



طراحی سخت افزار برای کاربردهای هوش مصنوعی

تمرین شماره 10

دانشکده مهندسی برق و کامپیوتر، دانشکدگان فنی دانشگاه تهران
نیم سال دوم سال تحصیلی 1404-1405

مقدمه:

در بسیاری از سامانه‌های هوش مصنوعی، شتاب‌دهنده‌های سخت‌افزاری تنها بخشی از کل سیستم هستند و بخش عمده منطق اجرای شبکه عصبی در نرم‌افزار انجام می‌شود. در چنین شرایطی لازم است نرم‌افزار بتواند با سخت‌افزار ارتباط برقرار کرده، داده‌ها را ارسال کند، عملیات را آغاز کند و نتایج را دریافت نماید.

در گام نخست لازم است با استفاده از کتابخانه *Cocotb* و *cocotbext-axi*، لایه‌ی راه‌انداز¹ نرم‌افزاری ایجاد کنید که بتوان از داخل برنامه‌های پایتون، شتاب‌دهنده پایه را مانند یک تابع معمولی فراخوانی کرد. همچنین در این پروژه میبایست عملیات نگاشت² و زمانبندی³ دو لایه *Conv2d* و *Linear* را به صورت نرم‌افزاری بر روی هسته انجام داده و با استفاده از راه‌انداز پیاده‌سازی شده بر روی هسته شبیه‌سازی کرده و در نهایت خروجی نهایی را با خروجی مدل طلایی⁴ مقایسه کنید.

در گام دوم این تمرین، شما با یک معماری پایه از یک شتاب‌دهنده شبکه عصبی آشنا می‌شوید و سپس با اعمال تکنیک‌های بهینه‌سازی در سطح معماری، آن را از حالت پردازش موازی (*Bit-Parallel*) به پردازش بیت‌سریال (*Bit-Serial*) ارتقا خواهید داد. در نهایت، با بهره‌گیری از تنگی داده‌ها در سطح بیت (*Bit-level sparsity*)، محاسبات بی‌اثر را حذف کرده و کارایی سیستم را بهبود می‌بخشید. همچنین با استفاده از محیط شبیه‌سازی بخش قبل، درستی سنجی انجام داده و نتایج را مقایسه کنید.

¹ Driver

² Mapping

³ Scheduling

⁴ Golden model

گام اول: شبیه‌سازی و درستی‌سنجی سیستم با Cocotb

آشنایی با Cocotb

Cocotb (Coroutine-based Co-simulation Testbench) یک فریم‌ورک متن‌باز برای نوشتن Testbench به زبان Python است. برخلاف Testbench‌های سنتی که با Verilog یا VHDL نوشته می‌شوند، در Cocotb کنترل کامل شبیه‌سازی در اختیار برنامه Python قرار می‌گیرد.

مزایای این روش عبارت‌اند از:

- تولید آسان داده‌های تست
- تحلیل نتایج با امکانات Python
- امکان مدل‌سازی سیستم‌های سطح بالا

برای اطلاعات بیشتر به سایت <https://www.cocotb.org> مراجعه فرمایید.

کتابخانه cocotbext-axi

نوشتن تراکنش‌های AXI به صورت دستی بسیار زمان‌بر است. به همین دلیل از کتابخانه *cocotbext-axi* استفاده می‌شود. این کتابخانه یک *AXI Master* کامل در اختیار برنامه قرار می‌دهد. برای اطلاعات بیشتر و مشاهده کدهای نمونه به آدرس <https://github.com/alexforenchich/cocotbext-axi> مراجعه فرمایید.

ساختار واسط‌ها

ماژول سخت‌افزاری دارای سه واسط *AXI* مستقل است.

واسط Configuration

- ارسال تنظیمات
- شروع عملیات
- خواندن وضعیت پایان اجرا

واسط Input & Weight

- ارسال ورودی‌ها
- ارسال وزن‌های لایه

واسط *Result*

- ارسال *Bias*
- دریافت خروجی محاسبه شده

خروجی مورد انتظار

همراه با صورت پروژه فایل اولیه کدهای شبیه ساز در اختیار شما قرار داده میشود. در پایان پروژه انتظار می رود بتوانید:

- یک راه انداز سخت افزاری مبتنی بر *Cocotb* طراحی کنید.
- ارتباط سه واسط *AXI* را مدیریت نمایید.
- ورودی و وزن لایه ورودی را چندی سازی کنید.
- لایه ورودی را نگاشت و زمان بندی کنید.
- ورودی هر بخش را برای سخت افزار ارسال کنید.
- پایان اجرای سخت افزار را تشخیص دهید.
- خروجی را دریافت کرده و به واحد زمان بند بازگردانید.
- خروجی نهایی را *DeQuantize* کنید.
- خروجی لایه شبیه سازی شده را با مدل طلایی مقایسه کنید.

نکته: برای پیاده سازی بخش های زمان بند و راه انداز می توانید الگوریتم های پیاده سازی شده در کدهای C را ملاحظه کرده و مشابه آن ها عمل کنید.

همچنین از طریق [این لینک](#) میتوانید به پروژه دسترسی داشته باشید.

گام دوم: معماری سخت افزاری شتاب دهنده

آشنایی با پلتفرم سخت افزاری فعلی (معماری Bit-Parallel)

پلتفرم فعلی، یک هسته پردازشی ساده برای محاسبات لایه های شبکه های عصبی (مانند *Fully Connected*) است که بر پایه جریان داده *Output Stationary* طراحی شده است. در این معماری، هر واحد پردازشی (*PE*) وظیفه محاسبه و انباشت مقدار نهایی یک خروجی را بر عهده دارد. تمامی ارتباطات سیستم با پردازنده میزبان (*Host*) از طریق پروتکل *AXI4* صورت می گیرد.

اجزای اصلی معماری پایه:

- حافظه‌های داده (*Input/Weight BRAMs*): این ماژول شامل حافظه‌های *BRAM* است که از طریق باس *AXI* مقداردهی می‌شوند. وزن‌ها به صورت بلوک‌های مجزا برای هر *PE* ذخیره شده و داده‌های ورودی در یک بلاک مشترک قرار دارند. همچنین با پیکربندی سیستم می‌توان تعداد این واحدهای محاسباتی را تعیین نمود.
- مسیر داده واحد *Cluster*: در هر سیکل کلاک، یک داده ورودی 8 بیتی از حافظه خوانده شده و به طور همزمان به تمامی واحدهای پردازشی ارسال (*Broadcast*) می‌شود. عملیات *MAC* (*Multiply-and-Accumulate*) داخل هر واحد پردازشی توسط یک ضرب‌کننده و یک جمع‌کننده انجام می‌گیرد. داده ورودی (مشترک) در همان سیکل در وزن 8 بیتی اختصاصی همان *PE* ضرب شده و نتیجه با مقدار 32 بیتی انباشت‌گر (*Accumulator*) جمع می‌شود.
- کنترلر واحد *Cluster*: این واحد شامل یک ماشین حالت بوده که اجزای سیستم را هماهنگ کرده و سیگنال‌های کنترلی لازم را برای واحدهای محاسباتی و حافظه‌های داده صادر می‌کند.

مقدمه‌ای بر محاسبات بیت‌سریال (*Bit-Serial Computation*)

در پیاده‌سازی سخت‌افزاری شبکه‌های عصبی عمیق (*DNNs*)، نحوه نمایش و پردازش عملوندها تأثیر مستقیمی بر مساحت تراشه، توان مصرفی و زمان اجرا دارد. در معماری فعلی (موازی)، تمام بیت‌های یک عملوند 8 بیتی در یک سیکل کلاک وارد ضرب‌کننده می‌شوند.

در محاسبات بیت‌سریال، عملوندها به جای پردازش موازی، بیت به بیت و طی چند سیکل متوالی پردازش می‌شوند. در شبکه‌های عصبی، معمولاً یک عملوند (مثلاً وزن) به صورت موازی (*Parallel*) ثابت نگه داشته می‌شود و عملوند دیگر (مثلاً ورودی) به صورت سریال شیفت داده می‌شود. در هر سیکل، تنها یک بیت از ورودی بررسی می‌شود:

- اگر بیت ورودی یک باشد، مقدار وزن (با در نظر گرفتن ارزش مکانی بیت ورودی، یعنی مقدار *Shift*) شیفت داده شده و با مقدار انباشت‌گر جمع می‌شود.
- اگر بیت ورودی 0 باشد، عملیات جمع صورت نمی‌گیرد و به سراغ بیت بعدی ورودی می‌رویم.

از جمله مزایای محاسبات *Bit-Serial* می‌توان به موارد زیر اشاره نمود:

1. کاهش مساحت و توان: جایگزینی ضرب‌کننده‌های کامل (که نیازمند گیت‌های منطقی زیادی هستند) با گیت‌های *AND*، شیفت‌رجیستر و جمع‌کننده.

2. بهره‌وری از تنگی داده‌ها در سطح بیت: در حالی که بیشتر شتابدهنده‌های شبکه عصبی قادر به حذف محاسبات غیرضرور در صورت صفر بودن کامل یک مقدار ورودی هستند (*Value-level sparsity*)، با استفاده از محاسبات سریال میتوان حتی از صفر بودن بیت‌ها در اپرندهای غیر صفر نیز برای افزایش بهره‌وری استفاده نمود.

از طرفی این روش معایب زیر را می‌تواند به همراه داشته باشد:

1. افزایش تاخیر (*Latency*): پردازش ضرب دو عدد 8 بیتی، اکنون به جای 1 سیکل، به 8 سیکل کلاک نیاز دارد. برای جبران این امر و رسیدن به *Throughput* مشابه محاسبات بیت‌موازی، در شتابدهنده‌های بیت‌سریال تعداد واحدهای محاسباتی را بیشتر می‌کنند.
2. پیچیدگی کنترلر: معمولاً این شتابدهنده‌ها نیاز به ماشین حالت‌های پیچیده‌تر هستند تا عملیات‌هایی از جمله استخراج بیت‌های ورودی و همگام‌سازی واحدهای محاسباتی را مدیریت کنند.

بخش 1: پیاده‌سازی پایه محاسبات سریال

در این گام باید معماری *Cluster*، واحد محاسباتی، و دیگر بخش‌های مورد نیاز را به گونه‌ای بازطراحی کنید که ورودی به صورت بیت‌سریال و وزن‌ها به صورت موازی پردازش شوند.

برای این کار لازم است که یک ماژول جدید طراحی کنید که ورودی را به صورت 8 بیتی از حافظه خوانده، آن را ذخیره کند و در هر سیکل، یک بیت از آن را به همراه سیگنال‌های کنترلی لازم به تمام PEها ارسال کند (واحد *Input Serializer*).

در مرحله بعد ضرب‌کننده‌های موازی پیشین باید با سخت‌افزار مناسب برای محاسبه بیت‌سریال (متشکل از گیت *AND* برای ضرب تک بیت ورودی در وزن، جمع‌کننده و عملگر شیفت) جایگزین شوند. در طراحی دقت شود که در محاسبات هر دو مقدار وزن و ورودی علامت دار می‌باشند.

همچنین توجه به اینکه عملیات ضرب یک داده 8 بیتی اکنون به 8 سیکل کلاک زمان نیاز دارد، ماشین حالت در ماژول *cluster_ctrl* باید به‌روزرسانی شود تا داده‌های جدید (ورودی بعدی و وزن‌های متناظر) دقیقاً پس از اتمام این 8 سیکل از حافظه واکنشی شوند. همچنین سیگنال‌های کنترلی لازم باید به واحدهای محاسباتی، واحد *Input serializer*، و حافظه‌ها صادر شوند.

موارد خواسته‌شده در این گام:

1. شماتیک مدارات طراحی شده: طراحی و رسم دقیق مسیر داده واحد محاسباتی بیت‌سریال و همچنین واحد کنترل ورودی (*Input Serializer*) که وظیفه پخش بیت‌به‌بیت ورودی را دارد، رسم کنید.
2. طراحی کنترلر: *State machine* کنترلر جدید را رسم کنید. دقت کنید که *FSM* باید به گونه‌ای تغییر کند که پس از واکنشی یک ورودی از حافظه، 8 سیکل در یک حالت میانی بماند تا عملیات بیت‌سریال تکمیل شود و سپس آدرس بعدی را تولید کند.
3. کدهای سخت‌افزاری: کدهای *Verilog* مورد نیاز را نوشته و یا ماژول‌های پیشین را در صورت نیاز اصلاح کنید.
4. تست واحد محاسباتی بیت‌سریال: یک *Testbench* جامع برای درستی سنجی واحد محاسباتی به تنهایی نوشته و نشان دهید که به ازای مجموعه‌ای از ورودی‌ها و وزن‌های مشخص، مقدار نهایی انباشت‌گرها در معماری بیت‌سریال دقیقاً با خروجی معماری موازی پایه تطابق دارد. (گزارش مقادیر مورد انتظار و شکل موج‌های مربوطه)
5. تست کامل سیستم در *Cocotb*: با استفاده از شبیه‌ساز توسعه داده شده در بخش قبل، سیستم بیت‌سریال جدید را درستی‌سنجی کنید. همچنین با اعمال یک لایه یکسان به عنوان ورودی به سیستم بیت‌سریال و بیت‌موازی، زمان اجرای آن را در دو حالت مقایسه کنید. برای مقایسه درست، تعداد واحدهای محاسباتی شتابدهنده بیت‌سریال را 8 برابر شتابدهنده پایه در نظر بگیرید.

بخش 2: بهینه‌سازی محاسبات (پرش از عملیات‌های بی‌اثر)

با توجه به ماهیت داده‌ها در شبکه‌های عصبی (مخصوصاً پس از اعمال توابعی مانند *ReLU*)، ورودی‌ها معمولاً دارای مقادیر صفر زیادی هستند، یا نمایش باینری آن‌ها شامل تعداد زیادی بیت صفر است. ضرب در صفر یک محاسبه بی‌اثر (*Ineffectual Computation*) است که صرفاً کلاک و توان مصرف می‌کند.

هدف این بخش، تبدیل تعداد کلاک‌های ثابت به کلاک‌های متغیر و وابسته به داده (*Data-dependent*) است. به این منظور، باید سیستم را به گونه‌ای تغییر دهید که تنها برای بیت‌های ورودی که مقدارشان 1 است، سیکل کلاک مصرف کند و محاسبات بی‌اثر را به طور کامل *Skip* کند.

نیازمندی‌های طراحی:

1. استخراج موقعیت‌های غیرصفر: واحد کنترل ورودی (طراحی شده در گام قبل) را به گونه‌ای تغییر دهید که تنها بیت‌های دارای ارزش 1 را پردازش کند. برای این کار باید از مدارهایی مانند

Leading-One Detector (LOD) یا *Priority Encoder* استفاده کنید تا ورودی ۸ بیتی به مجموعه‌ای از اندیس‌ها (توان‌های ۲) که مقدارشان ۱ است تبدیل شود.

2. اصلاح مسیر داده در واحد محاسباتی: واحد *PE* اکنون به جای دریافت مقدار بیت ۰ یا ۱، "ارزش مکانی (*Shift Amount*)" بیت یک را از کنترلر دریافت می‌کند. *PE* باید وزن را به اندازه این ارزش مکانی شیفت داده و با انباشت‌گر جمع کند. با این روش محاسبات بی‌اثر ضرب در بیت صفر به طور کلی *skip* میشوند.

3. زمان‌بندی متغیر: در این حالت، زمان اجرای عملیات دیگر ثابت و برابر با ۸ سیکل نخواهد بود. بلکه زمان اجرا دقیقاً برابر با تعداد بیت‌های ۱ (*Popcount*) در عدد ورودی است. کنترلر باید بتواند اتمام پردازش یک ورودی را تشخیص داده و بلافاصله واکنش داده بعدی را آغاز کند.

موارد خواسته شده در این گام:

1. شماتیک مدار استخراج‌کننده: مداری طراحی کنید که در هر سیکل، موقعیت ارزش‌دارترین بیت ۱ (یا کم‌ارزش‌ترین، بسته به طراحی شما) را در داده ورودی پیدا کند (استفاده از مدارهایی نظیر *Priority Encoder* یا *Leading-One Detector*). بلوک‌دیگرام کلی سیستم با در نظر گرفتن این مدار، واحدهای محاسباتی و سیگنال‌های میانشان را رسم کنید. در این بخش هم دقت کنید که ورودی و وزن هر دو علامت دار هستند و این موضوع را باید در طراحی لحاظ کنید. همچنین توجه شود که مدار طراحی شده در هر سیکل کلاک باید قادر باشد موقعیت یک بیت غیر صفر را استخراج کند. برای مثال اگر ورودی دارای ۵ بیت غیر صفر است، موقعیت هریک در یک سیکل استخراج شده و محاسبات در مجموعاً ۵ سیکل کامل میشود.
2. اصلاح مسیر داده در *PE*: در این حالت، *PE* به جای دریافت خود بیت (۰ یا ۱)، "ارزش مکانی" (*Shift Amount*) مربوط به آن بیت ۱ را دریافت می‌کند تا وزن را به همان اندازه شیفت داده و با انباشت‌گر جمع کند. شماتیک این تغییرات را ارائه دهید.
3. طراحی کنترلر زمان‌بندی متغیر: *State machine* کنترلر را به‌روز کنید. در این طراحی، زمان اجرای عملیات روی یک ورودی ۸ بیتی دیگر ثابت (۸ سیکل) نیست؛ بلکه برابر با تعداد بیت‌های ۱ در آن ورودی است. کنترلر باید بتواند اتمام پردازش یک ورودی را تشخیص داده و بلافاصله کلمه ورودی بعدی را لود کند.
4. تست واحد محاسباتی بیت‌سریال: یک *Testbench* همانند گام قبل برای درستی سنجی واحد محاسباتی به تنهایی نوشته و نشان دهید که به ازای مجموعه‌ای از ورودی‌ها و وزن‌های مشخص،

مقدار نهایی انباشت گرها در معماری بیت‌سریال دقیقاً با خروجی معماری موازی پایه تطابق دارد.
(گزارش مقادیر مورد انتظار و شکل موج‌های مربوطه)

5. تست کامل سیستم در *Cocotb*: با استفاده از شبیه‌ساز گام یک، همانند بخش قبل سیستم بیت‌سریال جدید را نیز درستی‌سنجی کنید. با اعمال همان لایه بخش قبل به عنوان ورودی به نسخه جدید سیستم بیت‌سریال، زمان اجرای آن را با دو حالت پیشین مقایسه کنید.

مواردی که باید تحویل دهید:

- پروژه در زمانی که بعداً اعلام خواهد شد به صورت مجازی تحویل گرفته خواهد شد.
- تمامی کدهای پیاده‌سازی شده شامل فایل‌های *Python* و *Verilog* آپلود شوند.
- گزارش کار شامل یافته‌ها و نتایج و تحلیل آن‌ها (لطفاً تمامی نکات و فرض‌هایی را که در پیاده‌سازی‌ها و محاسبات خود در نظر می‌گیرید در گزارش ذکر کنید)
- هدف از این تمرین، یادگیری شماسه! در صورت کشف تقلب، مطابق با قوانین درس برخورد خواهد شد.
- در صورت داشتن هرگونه سوال یا ابهام از طریق ایمیل زیر با دستیاران آموزشی در ارتباط باشید.

a.samoudi82@gmail.com

amirsadramasoudi@gmail.com

موفق باشید!